Scholarship Intelligence Crawler

Complete Build Guide - Edxso AI Engineer Intern, Assignment 2

Is guide ka maqsad: aapko step-by-step batana ki assignment ko **working system** ki tarah kaise build karna hai (Discover > Crawl > Extract > Verify > Score > Store > Update), sirf free tools use karke, aur evaluation ke har criteria ko kaise pass karna hai.

Evaluation criterion	Weight	Is guide mein kahan
Working crawler & automation	20%	Sections 5, 6, 12
Authenticity & official-source verification	25%	Sections 6, 8, 10
Accuracy of extracted information	20%	Sections 7, 9
Confidence / verification engine	15%	Section 8
Change & stale-data detection	10%	Section 10
Working UI / product output	5%	Section 11
Code quality	5%	Sections 3, 14

Sabse zaroori baat: Assignment mein 'fake / fabricated data' automatic failure hai. Is guide mein jahan bhi example data hai woh sirf *format dikhane* ke liye hai. Final database mein sirf wahi scholarships jayengi jo aapka crawler real official pages se nikaale. Kisi record ko haath se type karke mat daalna.

1. Project kya hai - ek paragraph mein

Aapko ek aisa system banana hai jo internet se Indian students ke liye scholarships dhundhe, unhe ek common schema mein convert kare, har scholarship ko **official source** se verify kare, ek **evidence-based confidence score** nikaale (LLM se number nahi poochna), database mein store kare, aur baar-baar chalne par changes (deadline change, scholarship hat gayi, naya aaya) detect kare. Saath mein ek dashboard chahiye jisme records, score ka reason, aur change history dikhe.

Minimum targets (checklist)

Requirement	Target	Kaise prove karenge
Real scholarship records	20+	scholarships table
Verified vs official source	15+	status = VERIFIED, source_type official
Confidence >= 95%	10+	confidence column + breakdown JSON
Different source types	3+	Govt portal, University, Corporate CSR / Foundation
Change detection examples	2+	change_log table (old/new/date/source/evidence)
Expired / stale examples	2+	status EXPIRED / NO_LONGER_VERIFIABLE
Auto re-run	Yes	run_crawl.py + scheduler (cron / GitHub Actions)

2. Tech stack (sab free)

Layer	Choice	Kyun
Language	Python 3.11+	Saare libraries available
HTTP crawling	requests + httpx, BeautifulSoup4, lxml	Simple, fast, robots.txt friendly
JS-heavy pages	Playwright (fallback only)	Kuch portals JS se render hote hain
PDF notices	pdfplumber	Government notifications aksar PDF hoti hain
Dates / amounts	dateparser + regex	Deterministic extraction, hallucination-free
LLM extraction	Ollama (llama3.1:8b / qwen2.5:7b) local	Free, offline. Alternative: HF transformers
Embeddings (optional)	sentence-transformers + FAISS	Duplicate detection / name matching
Database	SQLite	Single file, easily inspectable, repo mein commit ho sakta hai
API (optional)	FastAPI	Agar live API chahiye
Dashboard	Streamlit	Sabse tez UI. Streamlit Community Cloud pe free deploy
Scheduling	cron / GitHub Actions (scheduled workflow)	Auto-crawler ka proof

Agar laptop mein Ollama chalana mushkil ho to: pehle **sirf regex + rule based extraction** banao, LLM ko optional second pass rakho. Verification engine LLM ke bina bhi poora kaam karna chahiye.

3. Architecture aur folder structure

```
SEEDS (few portals + search queries)
       |
       v
[1 DISCOVER]  link-following + keyword scoring + search queries -> candidate URLs
       |
       v
[2 CLASSIFY]  domain rules: GOVT / UNIVERSITY / FOUNDATION / CSR / AGGREGATOR / UNKNOWN
       |            aggregator URL  -> resolve to the provider's official URL, else discovery-only
       v
[3 CRAWL]     fetch (robots.txt, rate limit, hash), HTML / PDF -> clean text snapshot
       |
       v
[4 EXTRACT]   regex/dateparser first, LLM (JSON schema) second; every field needs an evidence quote
       |
```

```
        v
 [5 VERIFY]    evidence grounding checks + score breakdown (weighted) + hard gates
        |
        v
 [6 STORE]     SQLite: scholarships, evidence, snapshots, change_log, crawl_runs
        |
        v
 [7 UPDATE]    diff vs previous, change_log rows, status transitions (EXPIRING_SOON, EXPIRED ...)
        |
        v
 STREAMLIT DASHBOARD (metrics, search, detail page, why-this-score, history, evidence)
```

```
scholarship-crawler/
  README.md
  requirements.txt
  config.yaml            # seeds, keywords, weights, thresholds
  db/schema.sql
  src/
    discover.py          # candidate URL discovery
    classify.py          # source-type classification
    fetch.py             # polite fetching + snapshots
    extract.py           # rule-based + LLM extraction with evidence
    verify.py            # scoring engine
    changes.py           # diff + status lifecycle
    store.py             # DB helpers
    run_crawl.py         # orchestrates one full run
  app/dashboard.py       # Streamlit UI
  data/scholarships.db   # generated database (commit sample)
  data/snapshots/        # raw page text per run (evidence archive)
  docs/technical_note.pdf
```

4. Setup commands

```
python -m venv .venv && source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install requests httpx beautifulsoup4 lxml pdfplumber dateparser pyyaml \
            streamlit pandas rapidfuzz tldextract python-dateutil
pip install playwright && playwright install chromium    # optional, JS pages
# optional LLM
curl -fsSL https://ollama.com/install.sh | sh
ollama pull llama3.1:8b
```

5. Database schema (SQLite)

Ek hi table mein sab kuch mat daalo. Evidence aur change history alag tables mein rakho taaki **Database -> Official Source -> Evidence -> Extracted Value** trace ho sake.

```
CREATE TABLE scholarships (
  id INTEGER PRIMARY KEY,
  fingerprint TEXT UNIQUE,           -- hash(normalised name + provider + official domain)
  name TEXT NOT NULL, provider TEXT,
  official_url TEXT, application_url TEXT,
  source_type TEXT,                  -- GOVERNMENT|UNIVERSITY|CORPORATE_CSR|FOUNDATION|INTERNATIONAL|NGO
  amount_text TEXT, amount_min INTEGER, amount_max INTEGER, benefit_type TEXT,
  eligibility_text TEXT,
  education_level TEXT, courses TEXT, academic_req TEXT,
  income_limit INTEGER,              -- NULL = Not specified (never guess)
  age_min INTEGER, age_max INTEGER, gender TEXT, categories TEXT, domicile_states TEXT,
  institution_req TEXT, documents_required TEXT, selection_process TEXT, renewal_req TEXT,
  opening_date TEXT, closing_date TEXT,        -- ISO yyyy-mm-dd or NULL
  status TEXT,                       -- ACTIVE|EXPIRING_SOON|EXPIRED|REVIEW_REQUIRED|NO_LONGER_VERIFIABLE
  verification_label TEXT,           -- VERIFIED | REVIEW REQUIRED
  confidence REAL, score_breakdown TEXT,       -- JSON: every check, points earned, reason
  first_seen TEXT, last_seen TEXT, last_verified TEXT, content_hash TEXT
);
CREATE TABLE evidence (
  id INTEGER PRIMARY KEY, scholarship_id INTEGER REFERENCES scholarships(id),
  field TEXT,            -- e.g. closing_date
  value TEXT,            -- extracted value
  quote TEXT,            -- exact sentence from page (must be substring of snapshot)
  source_url TEXT, snapshot_id INTEGER, extractor TEXT,   -- 'regex' | 'llm'
  grounded INTEGER       -- 1 if quote found verbatim in snapshot
);
CREATE TABLE snapshots (
  id INTEGER PRIMARY KEY, url TEXT, fetched_at TEXT, http_status INTEGER,
  content_hash TEXT, text_path TEXT, run_id INTEGER
);
CREATE TABLE change_log (
  id INTEGER PRIMARY KEY, scholarship_id INTEGER, field TEXT, old_value TEXT, new_value TEXT,
```

```
  detected_at TEXT, source_url TEXT, evidence_quote TEXT, change_type TEXT -- CHANGED|NEW|REMOVED|STATUS
);
CREATE TABLE crawl_runs (
  id INTEGER PRIMARY KEY, started_at TEXT, finished_at TEXT,
  discovered INTEGER, new INTEGER, changed INTEGER, expired INTEGER, notes TEXT
);
CREATE TABLE candidates (            -- discovery queue, incl. aggregator leads
  url TEXT PRIMARY KEY, discovered_via TEXT, source_class TEXT, score REAL, status TEXT, run_id INTEGER
);
```

6. Discovery + source classification (hard-coded scraper nahi)

Assignment explicitly kehta hai: *URL 1 -> scraper, URL 2 -> scraper* wala approach nahi chalega. Isliye ek **generic** crawler banao jo kisi bhi page ko same pipeline se guzare. Seeds sirf starting points hain.

Discovery ke 3 tareeke (teeno use karo)

- **Link-following with relevance scoring:** seed portal (jaise National Scholarship Portal, UGC, AICTE, ministry scholarship pages, university 'financial aid' pages) se links nikaalo. Anchor text + URL mein keywords (scholarship, fellowship, stipend, financial assistance, merit, NSP, award) dhundo, score do, top links queue mein daalo. Depth 2 tak jao.
- **Search-query discovery:** free search (DuckDuckGo HTML endpoint / SearXNG self-hosted) pe queries chalao: *'scholarship for Indian students site:gov.in'*, *'site:ac.in scholarship'*, *'CSR scholarship India foundation'*, *'scholarship notification 2026-27'*. Domain wise queries rotate karo.
- **Aggregator leads (sirf discovery):** Buddy4Study jaisi sites se scholarship ka naam + provider nikaalo, phir uske **official provider page** ko search/follow karke dhundo. Agar official page nahi mila to record REVIEW REQUIRED mein rahega, kabhi VERIFIED nahi.

Aggregator ka data kabhi field values ke source ke roop mein save mat karo. Aggregator se sirf 'lead' lo, values hamesha official page se extract karo. Yeh rule 'automatic failure' se bachata hai.

Source classification rules (classify.py)

Class	Rule	Trust
GOVERNMENT	domain endswith .gov.in / .nic.in / .gov / known central-state portals	High
UNIVERSITY	domain endswith .ac.in / .edu.in / .edu (+ page mentions university/institute)	High
FOUNDATION / CSR	Curated allowlist of known provider domains (Tata Trusts, Reliance Foundation, HDFC Parivartan, Aditya Birla, KC Mahindra ...) - allowlist config.yaml mein, alag verified	Medium-High
INTERNATIONAL	Official programme domains (e.g. Chevening, Fulbright-Nehru/USIEF)	Medium-High
AGGREGATOR	Blocklist: buddy4study, scholarships360-type sites, blogs, news, youtube, social	Discovery only
UNKNOWN	Everything else -> REVIEW REQUIRED path	Low

```
# src/classify.py
import tldextract
GOV = (".gov.in", ".nic.in")
EDU = (".ac.in", ".edu.in", ".edu")
AGGREGATORS = {"buddy4study.com", "scholarships.net.in", "medium.com", "blogspot.com", "youtube.com"}

def classify(url, allowlist):                    # allowlist: {domain: 'CORPORATE_CSR'|'FOUNDATION'|...}
    ext = tldextract.extract(url)
    host = ".".join(p for p in (ext.subdomain, ext.domain, ext.suffix) if p)
    reg = f"{ext.domain}.{ext.suffix}"
    if reg in AGGREGATORS:              return "AGGREGATOR", False
    if host.endswith(GOV):              return "GOVERNMENT", True
    if host.endswith(EDU):              return "UNIVERSITY", True
    if reg in allowlist:                return allowlist[reg], True
    return "UNKNOWN", False             # (class, is_official)

# src/discover.py  (core idea)
KEYWORDS = {"scholarship": 3, "fellowship": 3, "stipend": 2, "financial assistance": 2,
            "merit": 1, "notification": 1, "apply": 1, "eligibility": 1, "csr": 1}
def link_score(anchor, url):
    text = f"{anchor} {url}".lower()
    return sum(w for k, w in KEYWORDS.items() if k in text)

def discover_from_page(html, base_url, depth, max_depth=2):
    soup = BeautifulSoup(html, "lxml")
    out = []
    for a in soup.find_all("a", href=True):
        url = urljoin(base_url, a["href"]).split("#")[0]
        s = link_score(a.get_text(" ", strip=True), url)
```

```
        if s >= 3 and depth < max_depth:
            out.append((url, s))
    return sorted(set(out), key=lambda x: -x[1])[:50]
```

7. Crawling + extraction (anti-hallucination ke saath)

Fetching (fetch.py)

- robots.txt follow karo, har domain pe 2-3 sec delay, proper User-Agent, timeout + retry.
- Har fetch ka **snapshot** save karo: clean text + SHA-256 hash + timestamp + HTTP status. Yahi snapshot evidence ka base hai.
- HTML ke liye BeautifulSoup se nav/footer/script hatao. PDF ke liye pdfplumber. JS pages ke liye Playwright fallback.
- Agar page 404/410 aaye ya scholarship ka naam page se gayab ho - yeh 'removed' signal hai (Section 10).

Extraction pipeline (extract.py)

Rule: LLM ko sirf *reader* ki tarah use karo, *author* ki tarah nahi. Har extracted field ke saath page se ek **exact quote** maango. Fir code check kare ki woh quote snapshot text mein **verbatim** maujood hai. Nahi mila to field = **Not specified (NULL)**.

- **Pass A - deterministic:** regex + dateparser se closing date, opening date, amount (Rs./INR/per annum), income limit ('family income not exceeding...'), age, percentage marks. Jahan pattern milta hai wahan sentence quote ke roop mein save.
- **Pass B - LLM (Ollama):** page text ko JSON schema ke saath do. Prompt mein likho: 'Use only the given text. If a field is not stated, return null. For every non-null field return the exact supporting sentence.' temperature = 0.
- **Grounding check:** normalise whitespace, fir quote in page_text. Fail hone par value discard.
- **Cross-check:** Pass A aur Pass B ki values compare karo. Match = extraction-consistency points; mismatch = REVIEW.
- **Scholarship-page detection:** page ko record tab hi banao jab usme scholarship ka naam + (eligibility ya amount ya apply) signals ho. Listing/news pages ko skip karo.

```
# src/extract.py (core)
import re, dateparser, json
DATE_RX = re.compile(r"(last date|closing date|deadline|apply (?:on or )?before)[^.\n]{0,80}?"
                     r"(\d{1,2}[\s\-/.]\w+[\s\-/.]\d{2,4})", re.I)
INCOME_RX = re.compile(r"(?:family|annual|parental)\s+income[^.\n]{0,80}?(?:Rs\.?|INR|\u20b9)\s*([\d,\.]+)\s*(lakh|lac)?",

def norm(s): return re.sub(r"\s+", " ", s).strip().lower()

def grounded(quote, page_text):
    return bool(quote) and norm(quote) in norm(page_text)

def rule_extract(text):
    out = {}
    m = DATE_RX.search(text)
    if m:
        d = dateparser.parse(m.group(2), settings={"DATE_ORDER": "DMY"})
        if d: out["closing_date"] = (d.date().isoformat(), m.group(0))
    m = INCOME_RX.search(text)
    if m:
        n = float(m.group(1).replace(",", ""))
        n = int(n * 100000) if m.group(2) else int(n)
        out["income_limit"] = (n, m.group(0))
    return out                      # {field: (value, evidence_quote)}

def llm_extract(text, model="llama3.1:8b"):
    prompt = SCHEMA_PROMPT + "\nTEXT:\n" + text[:12000]
    r = requests.post("http://localhost:11434/api/generate",
        json={"model": model, "prompt": prompt, "format": "json", "stream": False,
              "options": {"temperature": 0}}).json()["response"]
    return json.loads(r)            # {field: {"value":..., "quote":...} or null}

def merge(rule, llm, text):
```

```
final = {}
for f in FIELDS:
    cand = rule.get(f) or (llm.get(f) and (llm[f]["value"], llm[f]["quote"]))
    if cand and grounded(cand[1], text): final[f] = cand      # else: stays Not specified
return final
```

8. Verification engine aur confidence score (sabse important)

Score ek **weighted checklist** se nikalta hai. Har check yes/no (ya fraction) hota hai jise code evaluate karta hai. LLM kabhi number nahi deta. Breakdown JSON mein save hota hai aur dashboard par 'Why this score?' mein dikhta hai.

#	Check (code se evaluate)	Points	Kaise measure
1	Source official hai	20	classify() -> official domain = 20, allowlisted provider = 16, else 0
2	Scholarship official page par abhi present hai	15	Fetch 200 AND fuzzy name match (rapidfuzz >= 90) page title/headings mein
3	Official application URL	10	Application link official domain par + HEAD request 200
4	Eligibility evidence-backed	15	(grounded eligibility fields) / (extracted eligibility fields) x 15
5	Deadline supported	10	Closing date quote grounded = 10; page explicitly says rolling/open = 8; absent = 0
6	Information current	5	Snapshot age <= 7 din = 5, <= 30 din = 3, warna 0; page 'last updated' bhi consider
7	Koi conflicting source nahi	10	Dusre official page/PDF se alag deadline/amount = 0, warna 10
8	Extraction consistent	10	Rule pass aur LLM pass key fields par agree: fraction x 10
9	Traceability coverage	5	Core fields (name, provider, amount, deadline, eligibility) mein se kitnon ka evidence row hai
	Total	**100**	

Hard gates (score kuch bhi ho)

- Source official nahi -> label kabhi VERIFIED nahi (max 60%).
- Name ya provider ka evidence quote grounded nahi -> REVIEW REQUIRED.
- Conflicting official sources -> REVIEW REQUIRED.
- Fetch fail / page missing -> status NO_LONGER_VERIFIABLE, confidence cap 40%.
- **VERIFIED sirf tab:** score >= 95.0 AND saare gates pass. Warna REVIEW REQUIRED.

```
# src/verify.py
def score(rec, ev, snap, conflicts, official, now):
    b = {}
    b["official_source"]  = (20 if official == "OFFICIAL" else 16 if official == "ALLOWLISTED" else 0,
                             f"source class {rec['source_type']}")
    b["present_on_source"] = (15 if snap["http_status"] == 200 and name_match(rec["name"], snap["text"]) >= 90 else 0,
                              "name found on official page")
    b["application_url"]  = (10 if rec["application_url"] and is_official(rec["application_url"])
                             and head_ok(rec["application_url"]) else 0, "official apply link reachable")
    elig = [e for e in ev if e["field"] in ELIG_FIELDS]
    ratio = sum(e["grounded"] for e in elig) / len(elig) if elig else 0
    b["eligibility"]      = (round(15 * ratio, 2), f"{sum(e['grounded'] for e in elig)}/{len(elig)} grounded")
    b["deadline"]         = (deadline_points(rec, ev), "closing date quote")
    b["freshness"]        = (freshness_points(snap["fetched_at"], now), "snapshot age")
    b["no_conflict"]      = (0 if conflicts else 10, f"{len(conflicts)} conflicts")
    b["consistency"]      = (round(10 * agreement_ratio(rec["_rule"], rec["_llm"]), 2), "rule vs LLM")
    b["traceability"]     = (round(5 * core_coverage(rec, ev), 2), "core fields with evidence")
    total = round(sum(v[0] for v in b.values()), 1)
    gates_ok = official != "NONE" and not conflicts and core_grounded(rec, ev, ["name", "provider"])
    label = "VERIFIED" if (total >= 95 and gates_ok) else "REVIEW REQUIRED"
    return total, label, b
```

Score ko **arbitrary** mat dikhne do: README + technical note mein yeh table paste karo, aur har record ka breakdown JSON dashboard par dikhao. Evaluator exactly yahi check karega.

9. Anti-hallucination - rules jo code mein enforce hone chahiye

- Koi bhi field tabhi store hoga jab uska **evidence quote snapshot mein verbatim** mile (grounded = 1).

- Missing information ka matlab hamesha **NULL / 'Not specified'**. Default ya 'typical' values kabhi nahi.
- LLM temperature 0, JSON-only output, 'use only given text' instruction, aur output post-validated.
- Amount ko text (amount_text) ke saath numeric form mein bhi store karo; numeric tabhi jab text mein number ho.
- Dates ko ISO mein parse karo; parse na ho to raw text rakho aur REVIEW flag lagao.
- Aggregator se kabhi field values copy mat karo.
- Dashboard par trace chain dikhao: **Record -> Official URL -> Evidence quote -> Extracted value**.
- Unit test banao: ek page jisme income limit nahi hai -> output income_limit IS NULL.

10. Change detection, stale detection, auto-run

Identity: record ko kaise pehchaanein

fingerprint = sha1(normalised_name + provider + official_domain). Isse Run 2 mein wahi scholarship dobara milne par naya record nahi banta - update/compare hota hai.

Har run ka flow (run_crawl.py)

```
def run():
    run_id = start_run()
    seen = set()
    for url in discover_candidates():                  # seeds + queue + search
        cls, official = classify(url, ALLOWLIST)
        snap = fetch(url, run_id)
        if not is_scholarship_page(snap.text): continue
        fields = merge(rule_extract(snap.text), llm_extract(snap.text), snap.text)
        fp = fingerprint(fields, url)
        seen.add(fp)
        prev = get_by_fingerprint(fp)
        if prev is None:
            insert_new(fields, snap, cls);  log(fp, "NEW")
        else:
            for f in TRACKED_FIELDS:                   # closing_date, amount, income_limit, status ...
                if prev[f] != fields.get(f):
                    log_change(prev.id, f, prev[f], fields.get(f), snap.url, quote(fields, f))  # old kept!
            update_record(prev.id, fields)
        rescore(fp)
    # anything previously ACTIVE but not seen / 404 this run:
    for rec in active_records():
        if rec.fingerprint not in seen: mark_missing(rec)    # see lifecycle below
    apply_date_rules()                                   # EXPIRING_SOON / EXPIRED
    finish_run(run_id)
```

Status lifecycle

Status	Rule
ACTIVE	Official page present, closing_date in future (> 15 din door) ya rolling
EXPIRING_SOON	closing_date <= 15 din door
EXPIRED	closing_date < today, ya official page par 'closed' / 'last date over' likha ho
REVIEW_REQUIRED	Conflict, score < 95, grounding fail, ya naya official notice contradict kare
NO_LONGER_VERIFIABLE	Official page 2 run tak missing/404 ya fetch fail (ek baar fail par sirf REVIEW_REQUIRED)

Change log entry (jo store hona chahiye)

```
CHANGE DETECTED  | scholarship: ABC Scholarship | field: closing_date
old: 2026-08-31  | new: 2026-09-15 | detected_at: <timestamp> | source: <official url>
evidence: "Last date to apply has been extended to 15 September 2026."
```

Assignment ke liye 2+ change aur 2+ expired examples kaise dikhayein

- **Expired:** Real data se hi aa jayega - purane cycles ki scholarships jinki closing_date nikal chuki hai automatically EXPIRED hongi. Aise 2+ records crawl mein rakho.
- **Changes (real):** Crawler ko 2 alag din chalao (e.g. Day 1 aur Day 3) aur GitHub Actions se schedule karo. Real change milna guaranteed nahi hota, isliye next option bhi tayyar rakho.
- **Changes (replay mode, honest):** Day 1 ke saved snapshots ko modify karke 'Run 2 fixture' banao (jaise deadline text badal do) aur --replay flag se chalao. Isko README + demo mein **clearly 'simulated change test'** label karo. Yeh engine ka test hai, real scholarship ka data nahi - isliye fabrication nahi maana jayega, bas chhupana mat.
- Wayback Machine ke purane snapshots se bhi 2 versions compare karke real change dikha sakte ho (agar archive accessible ho).

Auto-run scheduling

```
# .github/workflows/crawl.yml
on:
  schedule: [{cron: "30 2 * * *"}]      # roz subah
  workflow_dispatch:
jobs:
  crawl:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.11"}
      - run: pip install -r requirements.txt
      - run: python -m src.run_crawl
      - run: |
          git config user.name bot && git config user.email bot@example.com
          git add data/scholarships.db && git commit -m "crawl run" || true && git push
```

GitHub Actions pe Ollama nahi chalega. Wahan rule-based extraction chalao; LLM pass local machine par run karo. Ya dono modes config.yaml mein switch ke saath rakho.

11. Dashboard (Streamlit)

Assignment ka UI sirf 5% hai, isliye simple rakho par saari required cheezein dikhao.

Page / section	Kya dikhana hai
Overview	Total discovered, Verified, Review required, Active, Expired, Recently updated (last 7 din), Average confidence
List	Search box + filters (status, source type, label); columns: name, provider, deadline, status, confidence
Detail	Name, provider, amount, eligibility, deadline, official source link, application link, status
Why this score?	score_breakdown JSON ko table mein: check, points, reason
Change history	change_log rows: field, old, new, detected_at, source, evidence
Source evidence	evidence rows: field, extracted value, exact quote, URL, grounded tick

```
# app/dashboard.py
import streamlit as st, sqlite3, pandas as pd, json
con = sqlite3.connect("data/scholarships.db")
df = pd.read_sql("SELECT * FROM scholarships", con)
st.title("Scholarship Intelligence")
c = st.columns(6)
c[0].metric("Discovered", len(df))
c[1].metric("Verified", (df.verification_label == "VERIFIED").sum())
c[2].metric("Review required", (df.verification_label != "VERIFIED").sum())
c[3].metric("Active", (df.status == "ACTIVE").sum())
c[4].metric("Expired", (df.status == "EXPIRED").sum())
c[5].metric("Avg confidence", f"{df.confidence.mean():.1f}%")
q = st.text_input("Search")
view = df[df.name.str.contains(q, case=False) | df.provider.fillna("").str.contains(q, case=False)] if q else df
pick = st.selectbox("Open scholarship", view.name)
r = view[view.name == pick].iloc[0]
st.subheader(r["name"]); st.write(r["provider"], r["amount_text"], r["closing_date"], r["status"])
st.link_button("Official source", r["official_url"])
st.write("Why this score?"); st.dataframe(pd.DataFrame(json.loads(r["score_breakdown"])).T)
st.write("Change history"); st.dataframe(pd.read_sql("SELECT * FROM change_log WHERE scholarship_id=?", con, params=(int(r
st.write("Evidence"); st.dataframe(pd.read_sql("SELECT field,value,quote,source_url,grounded FROM evidence WHERE scholarsh
```

Run: streamlit run app/dashboard.py. Deploy: Streamlit Community Cloud (free) - repo connect karo, data/scholarships.db commit rakho taaki live app mein data dikhe.

12. 7-din ka build plan

Din	Kaam	Output
1	Repo setup, schema.sql, config.yaml, store.py, fetch.py (snapshots + robots + rate limit)	Ek URL fetch hoke snapshot save
2	classify.py + allowlist, discover.py (link scoring + search queries), candidates queue	50+ candidate URLs, classified
3	extract.py: regex/dateparser pass, grounding check, scholarship-page detector	10 pages se evidence-backed fields
4	Ollama LLM pass + merge + cross-check; unit tests (missing income -> NULL)	Consistent extraction
5	verify.py scoring + gates; score_breakdown; labels	Records pe confidence + VERIFIED/REVIEW
6	changes.py: diff, change_log, lifecycle, replay mode; Run 1 + Run 2	2+ changes, 2+ expired
7	Dashboard, README, technical note (3 pages), demo video, deploy	Submission ready

13. 20+ real records kaise banayein (manual inspection se pehle)

- Pehle 3 source types ke seeds lo: **Government** (National Scholarship Portal, UGC/AICTE/ministry scholarship pages), **University** (kuch .ac.in financial-aid / scholarship pages), **Corporate CSR / Foundation** (allowlisted provider sites). Seeds ko apni taraf se browser mein open karke confirm karo ki sahi aur official hain.

- Crawler jo mila use hi count karo. Agar koi record ki official page nahi mili to usko REVIEW REQUIRED rehne do. Quantity ke liye accuracy kurbaan mat karo.
- Run ke baad **khud 5-8 records manually** official page se match karo (name, deadline, amount). Jo galat nikle unki wajah dhundo aur extractor fix karo.
- Verified count kam aaye to scoring ko dheela mat karo. Extractor behtar banao ya pages ko behtar structure wale official sources se lo.
- Domains aur URLs ke bare mein: kisi bhi site ka exact URL ya scholarship ki date/amount main yahan guarantee nahi kar sakta, kyunki yeh time ke saath badalte hain. Hamesha apne crawler ke snapshot se verify karo.

14. Code quality aur submission checklist

Code quality

- Modules chhote aur single-purpose; type hints; docstrings; config.yaml mein weights/thresholds (code mein hard-code nahi).
- pytest tests: grounding, missing-field -> NULL, scoring gates, change detection, status lifecycle.
- Logging: har run ka summary (discovered/new/changed/expired) crawl_runs mein.
- requirements.txt pinned; .gitignore (venv, snapshots agar bade hon).

GitHub repo mein kya ho

- Source code, README (setup + run + architecture diagram), requirements.txt, schema.sql, config.yaml
- Sample data: data/scholarships.db aur CSV export
- docs/technical_note.pdf (max 3 pages), demo video link

Technical note (3 page) ka outline

Section	Kya likhna hai (sirf jo build kiya hai)
Architecture	Pipeline diagram + modules
Technology choices	Stack aur kyun (free/open-source)
Discovery methodology	Link scoring, search queries, aggregator-as-lead rule
Extraction methodology	Rule pass + LLM pass + evidence grounding
Verification methodology	Official-source classification, presence check, conflict check
Confidence methodology	Weighted table (Section 8) + hard gates + actual example breakdown
Anti-hallucination	Verbatim quote rule, NULL for missing, unit test
Change detection	Fingerprint, diff, change_log, lifecycle, real run vs simulated test (honestly)

Demo video (3-5 min) ka script

```
1. python -m src.run_crawl            -> logs: discovering, classifying, extracting
2. Show one scholarship being found   -> official source identified, aggregator rejected
3. Show evidence quote + score table  -> 'Why this score?'
4. Open dashboard                     -> metrics, search, detail page
5. Run crawl again (or --replay)      -> CHANGE DETECTED: old -> new, evidence
6. Show an EXPIRED / NO_LONGER_VERIFIABLE record
7. Answer: "Show us what your crawler found."
```

Automatic failure se bachne ka final check

Failure condition	Aapka bachav
Fabricated information	Grounding check + NULL for missing + manual spot-check

Failure condition	Aapka bachav
Official sources untraceable	official_url + evidence.source_url + snapshots
Arbitrary confidence	Weighted table + breakdown JSON + gates
Copying from aggregators	Aggregator = discovery only, never a value source
Only design doc / mock UI	Working crawler, populated DB, runnable dashboard
Paid services	Sab free: SQLite, Ollama, Streamlit Community Cloud, GitHub Actions

Agar aap chaho to next step mein main is guide ke hisaab se actual project code (repo structure, saari modules, dashboard) bana sakta hoon aur run karke dikha sakta hoon ki crawler real pages se kya nikalta hai.